Subject: Team Project

# Title: **Big Data Preprocessing of US Accidents (2016 - 2023)**


Mentor:
Slobodan Kalajdziski

Made by:
Ana Kolovska, 221226
Slobodanka Pishtolova, 211171
Georgi Bozhinoski, 216030
Damjan Janev, 211569


July, 2026

# Contents

# Abstract

This paper presents a machine learning pipeline for predicting the severity of traffic accidents in the United States using the US Accidents dataset (2016–2023), which contains approximately 7.7 million records collected across 49 states. The target variable, accident severity, is represented on a scale from 1 to 4, where higher values indicate greater impact on traffic flow.

The preprocessing phase involved removing redundant and high-missingness columns, engineering temporal features from accident timestamps, computing accident duration, grouping high-cardinality categorical variables such as weather conditions and wind direction, applying geographic clustering using MiniBatchKMeans to extract spatial patterns, and imputing missing values using median and mode strategies. Outlier capping was applied to weather-related numerical features based on physically reasonable domain bounds. Textual descriptions of accidents were incorporated as features through TF-IDF vectorization with bigrams to capture additional semantic information about accident context.

To address the severe class imbalance in the target variable — where Severity 2 accounts for nearly 80% of records — a combination of random undersampling and SMOTE oversampling was applied exclusively to the training set. A time-based split was used to prevent data leakage, with data up to 2020 used for training, 2021 for validation, and 2022 for testing.

Five classification models were evaluated: Logistic Regression, Random Forest, XGBoost, LightGBM, and Balanced Bagging Classifier. Models were assessed using Macro F1 score to ensure equal treatment of all severity classes. Random Forest achieved the highest Macro F1 of 0.493, followed closely by LightGBM at 0.491. All models struggled with Severity 1 prediction due to its extreme rarity in the dataset (0.87% of records), which represents an inherent limitation of the available data rather than a modeling failure.

# Introduction

Traffic accidents represent one of the most significant public safety challenges in the United States, resulting in tens of thousands of fatalities and millions of injuries annually. Beyond the human cost, traffic accidents impose substantial economic burdens through property damage, emergency response costs, healthcare expenditures, and lost productivity. Understanding the factors that contribute to accident severity is therefore of great importance to traffic management authorities, emergency responders, urban planners, and policymakers.

The emergence of large-scale traffic accident datasets, combined with advances in machine learning, presents an opportunity to develop predictive models that can identify patterns in accident severity based on environmental, temporal, and infrastructural factors. Such models could support proactive safety interventions, optimize emergency response allocation, and inform road infrastructure improvements.

This paper investigates the application of machine learning techniques to predict accident severity using the US Accidents dataset, a comprehensive collection of approximately 7.7 million accident records spanning the continental United States from 2016 to 2023. The dataset was compiled from multiple traffic data APIs and includes a rich set of features covering weather conditions, road characteristics, geographic location, time of occurrence, and natural language descriptions of each accident.

Predicting accident severity poses several notable challenges. First, the target variable is heavily imbalanced, with the majority of accidents classified as Severity 2, making it difficult for models to learn patterns associated with rare severity classes. Second, the dataset contains a substantial number of missing values across weather-related features, requiring careful imputation strategies. Third, several features exhibit high cardinality or require domain-specific preprocessing before they can be used effectively in machine learning models. Finally, the temporal nature of the data demands a time-aware train-test split to prevent data leakage and ensure that model evaluation reflects realistic deployment conditions.

To address these challenges, this paper proposes a comprehensive preprocessing and modeling pipeline that includes feature engineering, geographic clustering, text feature extraction, class balancing through SMOTE and undersampling, and evaluation using Macro F1 score. Five machine learning models are trained and compared, ranging from a linear baseline to ensemble tree-based methods, with the goal of identifying the most effective approach for this multiclass classification problem.

The remainder of this paper is organized as follows: Section 2 describes the dataset and preprocessing steps applied. Section 3 presents the feature engineering methodology. Section 4 outlines the experimental setup and models evaluated. Section 5 presents and discusses the results. Section 6 concludes the paper and outlines directions for future work.

# Dataset and Preprocessing

## Libraries Used

- pandas, numpy - data loading, manipulation, and numerical operations
- seaborn, matplotlib - visualizations: correlation heatmap, boxplots, class distribution bar chart, confusion matrices
- scikit-learn - preprocessing scalers, encoders, imputers, train/test split, and model evaluation metrics
- scikit-learn (additional) - TargetEncoder for high-cardinality columns, MiniBatchKMeans for geographic clustering, and TfidfVectorizer for text features from Description
- imbalanced-learn (imblearn) - SMOTE oversampling and RandomUnderSampler
- xgboost, lightgbm - GPU-accelerated gradient boosting classifiers

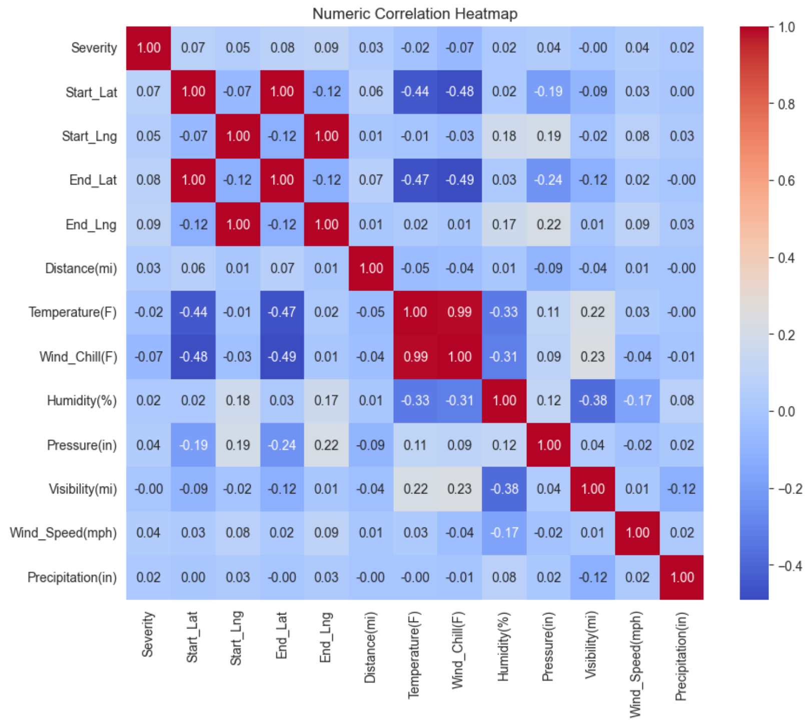
## Dataset Description

The dataset used in this study is the US Accidents dataset (2016–2023), a large-scale collection of traffic accident records covering 49 states of the United States. The dataset contains approximately 7.7 million records and 46 features, encompassing a wide range of information including geographic coordinates, weather conditions, road infrastructure characteristics, time of occurrence, and natural language descriptions of each accident. The target variable is Severity, an integer value ranging from 1 to 4, where 1 indicates the least impact on traffic flow and 4 indicates the most severe impact. The dataset was loaded into memory using the pandas library, occupying approximately 2.0 GB of RAM in its raw form.

## Data Cleaning and Column Removal

The first preprocessing step involved identifying and removing columns that were either redundant, uninformative, or contained an excessive proportion of missing values.
The following columns were dropped entirely: ID and Source are administrative identifiers that carry no predictive value. End_Lat and End_Lng were removed due to 44% missing values, making reliable imputation infeasible. Wind_Chill(F) was dropped as it is a derived quantity already captured by the combination of temperature and wind speed. Precipitation(in) was removed due to 28.5% missing values, with precipitation conditions already represented through the weather condition grouping. Country was constant across all records. Timezone, Airport_Code, Zipcode, and Weather_Timestamp were removed as administrative or redundant identifiers. Civil_Twilight, Nautical_Twilight, and Astronomical_Twilight were dropped as they are all highly correlated with Sunrise_Sunset, which was retained. Turning_Loop was removed due to near-zero variance. Street was dropped due to extremely high cardinality with limited predictive utility.

Additionally, records with negative accident duration or duration exceeding 2880 minutes (48 hours) were removed as physically implausible, resulting in a minor reduction in dataset size.




*1. The data correlation at the beginning*

## Feature Engineering

Several new features were engineered from existing columns to better capture temporal and spatial patterns in the data.
*Temporal Features:* The Start_Time column was parsed into its constituent components to extract the following features: Hour (hour of day, 0–23), Day_of_week (day of the week, 0=Monday, 6=Sunday), Month (month of year, 1–12), and Year (calendar year, used for time-based splitting). Two binary indicator features were also derived: Is_rush_hour, set to 1 for hours 7–9 and 16–18, and Is_weekend, set to 1 for Saturday and Sunday. The original Start_Time column was dropped after feature extraction.

*Accident Duration:* A new feature Accident_Duration was computed as the difference between End_Time and Start_Time in minutes. This feature captures the operational impact of each accident and is expected to correlate with severity, as more severe accidents tend to result in longer road disruptions.

*Geographic Clustering:* Raw latitude and longitude coordinates, while informative, are difficult for tree-based models to interpret as continuous values across a continental scale. To address this, MiniBatchKMeans clustering with 200 clusters was applied to the Start_Lat and Start_Lng columns, producing a Geo_Cluster feature that groups accidents into geographic regions. These clusters implicitly capture spatial semantics such as high-traffic intersections, highway corridors, and urban density zones without requiring manual labeling.

## Categorical Feature Grouping

Two high-cardinality categorical columns required grouping before encoding.
*Weather Condition:* The Weather_Condition column contained over 130 unique values representing fine-grained weather descriptions. These were manually mapped into seven semantically meaningful groups: Clear, Cloudy, Rain, Snow, Fog, Dust/Sand, and Storm. Any unmapped values were assigned to an Other category, which in practice contained only null values.

*Wind Direction:* The Wind_Direction column contained compass direction abbreviations and full names across multiple formats. These were consolidated into ten directional groups: North, Northeast, East, Southeast, South, Southwest, West, Northwest, Calm, and Variable. Missing wind direction values were filled with the mode of the grouped column to avoid row loss.

## Missing Value Imputation

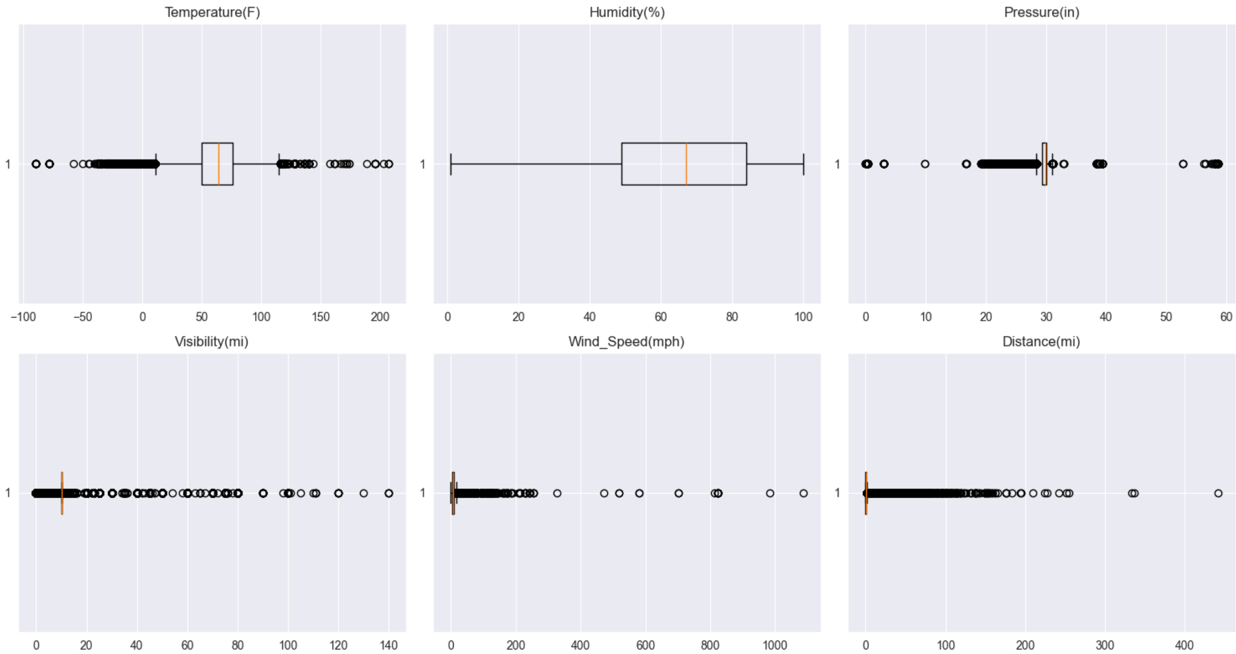
After column removal and feature engineering, the remaining missing values were addressed as follows. Numerical weather sensor columns — Temperature(F), Humidity(%), Pressure(in), Visibility(mi), and Wind_Speed(mph) — were imputed using the column median, which is robust to the skewed distributions present in weather data. The Sunrise_Sunset column was imputed using the mode, as it is a binary categorical feature (Day/Night). The City column, with only 253 missing values out of 7.7 million records, was filled with the placeholder value 'Unknown'. After imputation, five residual missing values remained in the Description column, which were handled downstream by treating them as empty strings during TF-IDF vectorization.

| | |
|---|---|
| Description | 0.00 |
| City | 0.00 |
| Temperature(F) | 2.12 |
| Humidity(%) | 2.25 |
| Pressure(in) | 1.82 |
| Visibility(mi) | 2.29 |
| Wind_Speed(mph) | 7.39 |
| Sunrise_Sunset | 0.30 |

*2. Percentage of missing data per column (only columns with missing data)*

## Outlier Treatment

Outlier analysis was performed on six numerical columns using the interquartile range (IQR) method. Based on this analysis and domain knowledge, capping (Winsorization) was applied to three columns containing physically implausible values. Temperature(F) was capped at −50°F and 130°F, covering the full realistic range of US surface temperatures. Pressure(in) was capped at 27.0 and 32.0 inHg, corresponding to the valid range of atmospheric pressure at sea level. Wind_Speed(mph) was capped at 0 and 74 mph, the threshold for Category 1 hurricane-force winds. Visibility(mi) and Distance(mi) were left uncapped as their distributions, while flagged by the IQR method, reflect genuine real-world skewness rather than data errors. Humidity(%) required no treatment as it contained no statistical outliers.



*3. Data outliers*

## Encoding

Categorical features were encoded using strategies appropriate to their cardinality and nature. Boolean columns representing road infrastructure features — Amenity, Bump, Crossing, Give_Way, Junction, No_Exit, Railway, Roundabout, Station, Stop, Traffic_Calming, and Traffic_Signal — were converted from boolean to integer (0/1).

Low-cardinality categorical columns — Sunrise_Sunset, weather_group, and Wind_Group — were encoded using one-hot encoding with the first category dropped to avoid multicollinearity, producing binary indicator columns for each category.

High-cardinality columns — City, County, State, and Geo_Cluster — were encoded using target encoding, replacing each category with the mean value of the target variable (Severity) for that category as observed in the training set. Target encoding was fitted exclusively on training data and applied to validation and test sets separately to prevent data leakage.

## Text Feature Extraction

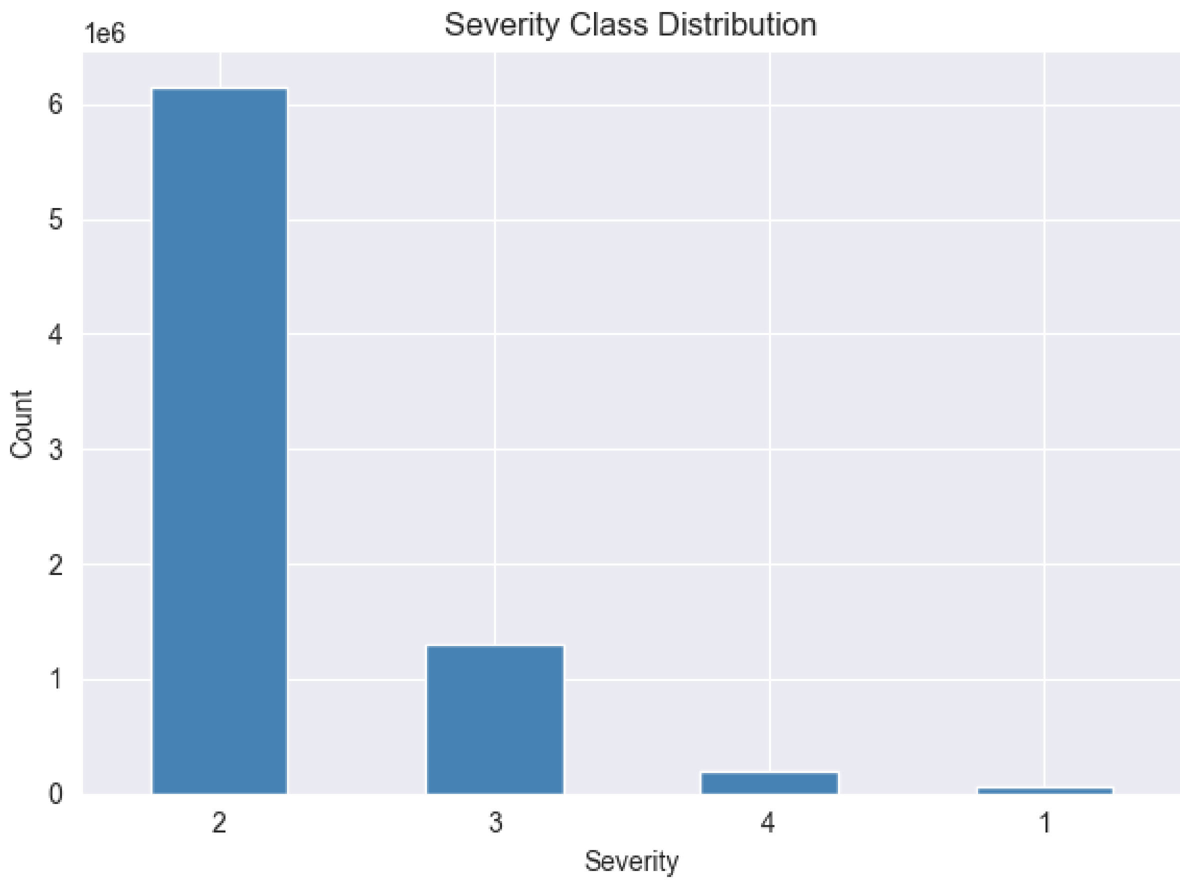
The Description column contains free-text descriptions of each accident, such as road names, directions, and incident types. To extract predictive signal from this text, TF-IDF (Term Frequency-Inverse Document Frequency) vectorization was applied with the following configuration: a maximum of 40 features, English stop word removal, bigram range (1–2 words), and a minimum document frequency of 50. The resulting 40 TF-IDF features were appended to the structured feature matrix, bringing the total feature count to 87. TF-IDF was fitted on training descriptions only and applied to validation and test sets separately.

## Train/Validation/Test Split

Given the temporal nature of the dataset, a time-based split was used instead of random sampling to prevent data leakage. Records from 2016 to 2020 were used for training (4,151,593 records), 2021 for validation (1,558,723 records), and 2022 for testing (1,760,749 records). This ensures that the model is always evaluated on data from a later time period than it was trained on, accurately reflecting real-world deployment conditions where only historical data is available at prediction time.

## Class Imbalance Handling

The target variable exhibits severe class imbalance, with Severity 2 accounting for approximately 79.7% of all records, while Severity 1 represents only 0.87%. Training a model directly on this distribution would result in a strong bias toward the majority class.
To address this, a two-stage resampling pipeline was applied exclusively to the training set. First, random undersampling was used to reduce the Severity 2 class from 2,926,701 to 1,000,000 samples. Second, SMOTE (Synthetic Minority Oversampling Technique) was applied to generate synthetic samples for the minority classes, bringing Severity 1 to 150,000 samples and Severity 4 to 500,000 samples. Severity 3, with 1,073,137 samples, was left unchanged. The final resampled training set contained 2,723,137 records distributed across all four severity classes. Resampling was not applied to the validation or test sets, which retain the natural class distribution to enable realistic evaluation. Additionally, all models were trained with class_weight='balanced' to further penalize misclassification of minority classes.




*4. Severity data imbalance*

# Experimental Setup and Models

## Evaluation Metric

Given the severe class imbalance in the target variable, accuracy alone is an insufficient evaluation metric. A model that predicts Severity 2 for every record would achieve approximately 80% accuracy while being entirely useless for identifying other severity classes. Therefore, the primary evaluation metric used in this study is Macro F1 score, which computes the F1 score independently for each class and averages them with equal weight regardless of class size. This ensures that performance on rare classes such as Severity 1 is penalized equally to performance on the majority class.

The F1 score for each class is defined as the harmonic mean of precision and recall:
$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$
where Precision measures the proportion of predicted positives that are truly positive, and Recall measures the proportion of actual positives that were correctly identified. The Macro F1 score is then computed as the unweighted average of per-class F1 scores across all four severity levels.

## Models

Five classification models were trained and evaluated in this study, ranging from a linear baseline to ensemble tree-based methods.
*Logistic Regression* was included as a linear baseline to establish a lower bound on performance. It models the probability of each class as a softmax function of a linear combination of input features. Despite its simplicity, it provides a useful reference point for assessing the benefit of more complex models. The model was configured with a maximum of 2000 iterations using the lbfgs solver and balanced class weights.

*Random Forest* is an ensemble method that trains a large number of decision trees on random subsets of the data and features, aggregating their predictions through majority voting. It is robust to overfitting and handles mixed feature types well. In this study, 100 trees were trained with a maximum depth of 20, minimum samples per split of 50, and minimum samples per leaf of 20, with balanced class weights applied.

*XGBoost* (Extreme Gradient Boosting) is a gradient boosting framework that builds trees sequentially, with each tree correcting the errors of the previous one. It is widely regarded as one of the most effective algorithms for tabular data classification tasks. The model was configured with 200 estimators, a maximum depth of 6, a learning rate of 0.1, and subsampling rates of 0.8 for both rows and columns. Training was performed on GPU using CUDA to accelerate computation. Sample weights were computed using the balanced weighting strategy to account for class imbalance.

*LightGBM* (Light Gradient Boosting Machine) is a gradient boosting framework optimized for large datasets through histogram-based tree construction and leaf-wise tree growth. It is generally faster than XGBoost on large datasets while achieving comparable accuracy. The model was configured with 200 estimators, a maximum depth of 10, 63 leaves per tree, a learning rate of 0.1, and subsampling rates of 0.8. Training was performed on GPU using OpenCL acceleration via the device='gpu' parameter, and balanced class weights were applied.

*Balanced Bagging Classifier* is an ensemble method that combines bagging with random undersampling. Each base estimator in the ensemble is trained on a randomly undersampled balanced subset of the training data, making it inherently suited for imbalanced classification problems without requiring explicit resampling of the full dataset. The model was configured with 15 base estimators and trained directly on the original (non-resampled) training set, unlike the other models which were trained on the SMOTE-resampled data.

## Hardware and Software

All experiments were conducted on a system equipped with an NVIDIA GeForce RTX 3060 Ti GPU, which was utilized for XGBoost and LightGBM training. The software stack consisted of Python 3.13, scikit-learn for Logistic Regression, Random Forest, and Balanced Bagging, xgboost and lightgbm libraries for the respective gradient boosting models, and imbalanced-learn for SMOTE and resampling pipelines. Data manipulation was performed using pandas and numpy, and visualizations were produced using matplotlib and seaborn.

## Experimental Procedure

Each model was trained on the resampled training set containing 2,723,137 records and 87 features, with the exception of Balanced Bagging which was trained on the original unbalanced training set of 4,151,593 records. Predictions were generated on the held-out 2022 test set containing 1,760,749 records. Training time was recorded for each model to assess computational efficiency alongside predictive performance. For each model, the following metrics were computed on the test set: overall accuracy, Macro F1 score, per-class precision, recall, and F1 score, and a confusion matrix showing the distribution of predicted versus actual severity classes.

## Results and Discussion

### Overall Model Performance

Table 1 summarizes the Macro F1 score, accuracy, and training time for each of the five models evaluated on the 2022 test set.

| Model | Macro F1 | Accuracy | Training Time |
|---|---|---|---|
| Random Forest | 0.493 | 85.9% | 180.3s |
| LightGBM | 0.491 | 87.2% | 83.7s |
| Logistic Regression | 0.485 | 82.8% | 1228.2s |
| XGBoost | 0.479 | 85.0% | 21.9s |
| Balanced Bagging | 0.328 | 68.8% | 116.8s |

Random Forest achieved the highest Macro F1 score of 0.493, followed closely by LightGBM at 0.491. Despite having the highest overall accuracy at 87.2%, LightGBM ranks second in Macro F1, illustrating the disconnect between accuracy and balanced class performance in imbalanced classification tasks. XGBoost achieved the fastest training time among competitive models at 21.9 seconds, benefiting from GPU acceleration, while delivering a Macro F1 of 0.479. Logistic Regression, despite being the simplest model, achieved a competitive Macro F1 of 0.485, though at a significantly higher computational cost of over 20 minutes. Balanced Bagging performed the worst across all metrics, suggesting that its undersampling-based approach was insufficient for capturing the complexity of this dataset.

The relatively narrow range of Macro F1 scores among the top four models (0.479–0.493) suggests that the primary limitation on performance is not the choice of model but rather the inherent difficulty of the classification problem, particularly with respect to Severity 1.

### Per-Class Performance Analysis

A more detailed picture emerges when examining per-class precision, recall, and F1 scores for each model.

*Severity 2* was predicted well by all models, with F1 scores ranging from 0.81 (Balanced Bagging) to 0.93 (LightGBM). This is expected given that Severity 2 represents nearly 80% of the test set, providing the models with abundant training signal.

*Severity 3* was also predicted reasonably well, with recall values exceeding 0.90 across most models. Random Forest achieved near-perfect recall of 0.98 for this class, though at the cost of lower precision (0.38), indicating that it over-predicts Severity 3. LightGBM achieved the best balance with a Severity 3 F1 of 0.60.

*Severity 4* showed moderate performance across models, with F1 scores ranging from 0.16 (Balanced Bagging) to 0.49 (Random Forest). Most models achieved reasonable recall for this class (0.75–0.87), meaning they successfully identified the majority of severe accidents, though precision remained low, indicating a tendency to over-predict Severity 4.

*Severity 1* was the most problematic class across all models, with F1 scores near zero for most approaches. The best performing model for this class was Logistic Regression with an F1 of 0.12, while tree-based models achieved F1 scores between 0.01 and 0.05. This failure can be attributed primarily to the extreme rarity of Severity 1 in the training data — only 29,337 genuine samples out of 4.15 million training records (0.71%). Even after SMOTE oversampling to 150,000 samples, the synthetic nature of the generated samples was insufficient to teach models to reliably identify this class on real test data.

### Impact of Class Imbalance Handling

The combination of SMOTE oversampling and random undersampling meaningfully improved model performance on minority classes compared to training on the raw imbalanced data. The addition of class_weight='balanced' further reinforced this effect. However, the results demonstrate that these techniques have practical limits when the minority class is as severely underrepresented as Severity 1 in this dataset.

Balanced Bagging, which uses a different imbalance handling strategy based on undersampling within each bagging iteration, performed significantly worse than the other models. This suggests that for this dataset, the combination of SMOTE with explicit class weighting is a more effective strategy than undersampling-based ensemble methods.

### Impact of Text Features

The inclusion of TF-IDF features derived from the Description column added 40 features to the model input, bringing the total feature count from 47 to 87. These features capture linguistic patterns in accident descriptions such as road names, incident types, and directional information that are not captured by structured features alone. While a direct ablation study comparing models with and without text features was not conducted in this work, the inclusion of description-based features is expected to have contributed positively to model performance, particularly for distinguishing between severity levels that share similar structured feature profiles.

### Computational Efficiency

From a practical standpoint, XGBoost stands out as the most computationally efficient model, completing training in 21.9 seconds on GPU while achieving competitive performance. LightGBM required 83.7 seconds on GPU, and Random Forest took 180.3 seconds on CPU. Logistic Regression was by far the most expensive model to train at over 20 minutes, and still failed to converge within the allocated iteration budget, making it impractical for this dataset size. For deployment scenarios where retraining frequency is important, XGBoost offers the best trade-off between speed and predictive performance.

### Limitations

Several limitations of this study are worth acknowledging. First, the near-complete failure to predict Severity 1 reflects a fundamental data scarcity issue that cannot be fully resolved through resampling alone. A larger collection of genuine Severity 1 records would be necessary to meaningfully improve performance on this class. Second, the Macro F1 scores of approximately 0.49 across all competitive models suggest that the current feature set, while comprehensive, may not fully capture all factors that determine accident severity. Third, the dataset relies on data collected from traffic APIs which may introduce reporting biases — certain regions or accident types may be systematically over or underrepresented. Finally, the model was evaluated on a single held-out year (2022), and performance may vary across different time periods due to shifts in traffic patterns, reporting practices, or road conditions.

# Conclusion and Future Work

## Conclusion

This paper presented a comprehensive machine learning pipeline for predicting the severity of traffic accidents in the United States using a large-scale dataset of approximately 7.7 million records. The pipeline addressed several key challenges inherent to this problem, including severe class imbalance, high-cardinality categorical features, missing weather sensor data, temporally structured observations, and the availability of unstructured text descriptions.

The preprocessing pipeline involved removing redundant and high-missingness columns, engineering temporal features from accident timestamps, computing accident duration, grouping weather conditions and wind directions into semantically meaningful categories, applying geographic clustering to extract spatial patterns, imputing missing values using median and mode strategies, capping physically implausible outliers using domain knowledge, and encoding categorical features using one-hot and target encoding. Text features were extracted from accident descriptions using TF-IDF vectorization with bigrams, contributing an additional 40 features to the model input. A time-based train/validation/test split was used to prevent data leakage and simulate realistic deployment conditions.

Five classification models were trained and evaluated using Macro F1 score as the primary metric. Random Forest achieved the highest Macro F1 of 0.493, followed closely by LightGBM at 0.491. XGBoost demonstrated the best computational efficiency, completing training in under 22 seconds on GPU while achieving a competitive Macro F1 of 0.479. Balanced Bagging performed the worst, suggesting that undersampling-based ensemble methods are insufficient for this problem. Logistic Regression, while achieving a reasonable Macro F1 of 0.485, required over 20 minutes to train and failed to converge, making it impractical at this scale.

Across all models, Severity 1 prediction remained the most significant challenge, with F1 scores near zero for tree-based methods and only 0.12 for Logistic Regression. This is attributable to the extreme rarity of Severity 1 records in the dataset — less than 1% of all records — which limits the effectiveness of synthetic oversampling techniques such as SMOTE. This finding highlights a fundamental data collection limitation that cannot be resolved through modeling choices alone.

Overall, the results demonstrate that machine learning models can effectively distinguish between moderate and severe accident conditions, particularly for Severity 2 and Severity 3, but struggle with the extremes of the severity scale due to data imbalance. The combination of SMOTE oversampling, random undersampling, and balanced class weights proved to be the most effective strategy for mitigating class imbalance within the constraints of the available data.

## Future Work

Several directions for future research could build upon the findings of this study.

*Improved minority class handling:* The most pressing limitation is the poor prediction of Severity 1. Future work could explore more sophisticated oversampling techniques such as ADASYN (Adaptive Synthetic Sampling), which generates more synthetic samples in regions of the feature space where the classifier performs poorly. Alternatively, a cost-sensitive learning framework that assigns asymmetric misclassification costs could be explored to further penalize Severity 1 errors.

*Hyperparameter tuning:* The models in this study were trained with manually selected hyperparameters. Systematic hyperparameter optimization using frameworks such as Optuna or Bayesian optimization could yield meaningful improvements in Macro F1, particularly for XGBoost and LightGBM which are sensitive to tree depth, learning rate, and regularization parameters.

*Advanced text feature extraction:* The TF-IDF approach used in this study is a relatively simple text representation. Future work could explore the use of pre-trained language models such as BERT or sentence transformers to generate richer semantic embeddings from accident descriptions, potentially capturing more nuanced information about accident context and severity.

*Feature importance analysis:* A systematic analysis of feature importances from tree-based models could identify which features contribute most to severity prediction and guide future data collection efforts. Features with low importance could be dropped to simplify the model, while high-importance features could inform targeted data quality improvements.

*Temporal modeling:* The current approach treats each accident as an independent record. Future work could explore temporal modeling approaches that account for time-series patterns in accident occurrence, such as recurrent neural networks or temporal convolutional networks, which may capture cyclical patterns in accident severity that are not captured by static features alone.

*Geographic granularity:* The geographic clustering approach used in this study groups accidents into 200 clusters across the continental United States. Future work could explore finer-grained geographic representations, such as road segment identifiers or census tract features, to capture more localized spatial patterns in accident severity.

*Cross-year validation:* This study evaluated models on a single held-out year (2022). Future work could assess model stability across multiple years using rolling time-based cross-validation, providing a more robust estimate of generalization performance and identifying potential concept drift over time as traffic patterns and reporting practices evolve.

*Multimodal data integration:* Future work could enrich the dataset with additional data sources such as real-time traffic volume data, road surface conditions, speed limit information, and historical accident frequency by road segment, all of which are likely to have strong predictive value for accident severity.

## Links

Github: https://github.com/georgibozhinoski/US-Accidents-2016---2023---Big-Data-Preprocessing/tree/master

Dataset: https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents

Libraries:


1. Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*. https://scikit-learn.org
2. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. https://arxiv.org/abs/1603.02754
3. Ke, G. et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html
4. Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn. *JMLR*. https://imbalanced-learn.org
5. Harris et al. (2020). Array programming with NumPy. https://numpy.org
6. McKinney, W. (2010). Data Structures for Statistical Computing in Python (pandas). https://pandas.pydata.org
7. Hunter, J. D. (2007). Matplotlib. https://matplotlib.org
8. Waskom, M. (2021). Seaborn. https://seaborn.pydata.org


Methods:


1. Chawla, N. V. et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *JAIR*. https://arxiv.org/abs/1106.1813
2. Breiman, L. (2001). Random Forests. *Machine Learning*. https://link.springer.com/article/10.1023/A:1010933404324